Technical Documentation for the ELEN3020A Group 3 Online Morabaraba Multiplayer Platform

ELEN3020A – Group 3

11 May 2026

Karen Cheng (2658083)
Akiria Chetty (2801838)
Joanna Chronis (2817173)

1. Introduction

This document provides a comprehensive technical overview of a real-time online multiplayer Morabaraba game platform developed in Unity, as part of the ELEN3020A semester project. The project implements two traditional strategy games—Morabaraba and Six Men's Morris—adapted into a fully networked digital environment that supports account creation, lobby making, synchronised turn-based gameplay, and persistent player statistics.

The system is designed around a client–host multiplayer architecture using Unity's Netcode for GameObjects and Unity Relay, enabling low-latency gameplay while still maintaining secure and synchronised game states across all connected devices. In addition to core gameplay functionality, the platform integrates cloud-based services for authentication, data persistence, and lobby management.

The development process focused on modular system design, separating responsibilities across three main sections: gameplay logic, user interface management and networking, and data-handling systems.

This document outlines the technologies used, system architecture, core gameplay and support systems, design decisions, challenges encountered during development, testing suites, and a summary of total project time.

2. Technologies used

- **Unity Game Engine**

  Unity Game Engine was used to run and compile all the C# codes, assets and packages to create a functional online multiplayer application.

  - Unity Services Core was used for accessing Unity Cloud services.
  - Unity Multiplayer Services contains all the packages for an online multiplayer platform.
  - Unity Relay acts as a safety guard for players to connect to the same lobby without exposing their IPs.

- Unity Netcode for GameObjects was used to synchronise state, game objects and scenes between players in real-time.
- Unity Transport is the low-level, high-performance networking library that moves data between the host and guest.

- **Visual Studio**

Visual Studio code was utilised to create all the C# scripts needed for the platform to function. Visual Studio code has a built-in relationship with Unity, thus making it the best option for the task at hand.

- **Github**

Github was used for version control and team member collaboration, with each member having their own dedicated branch. This allowed team members to effectively push and pull changes to ensure all members had up-to-date versions of the project.

- **Discord**

Discord was the main mode of communication between members, as it allowed for voice chats, message sharing, screen sharing and interactivity logs. This allowed the team to remain connected and in contact for decisions that affected all members.

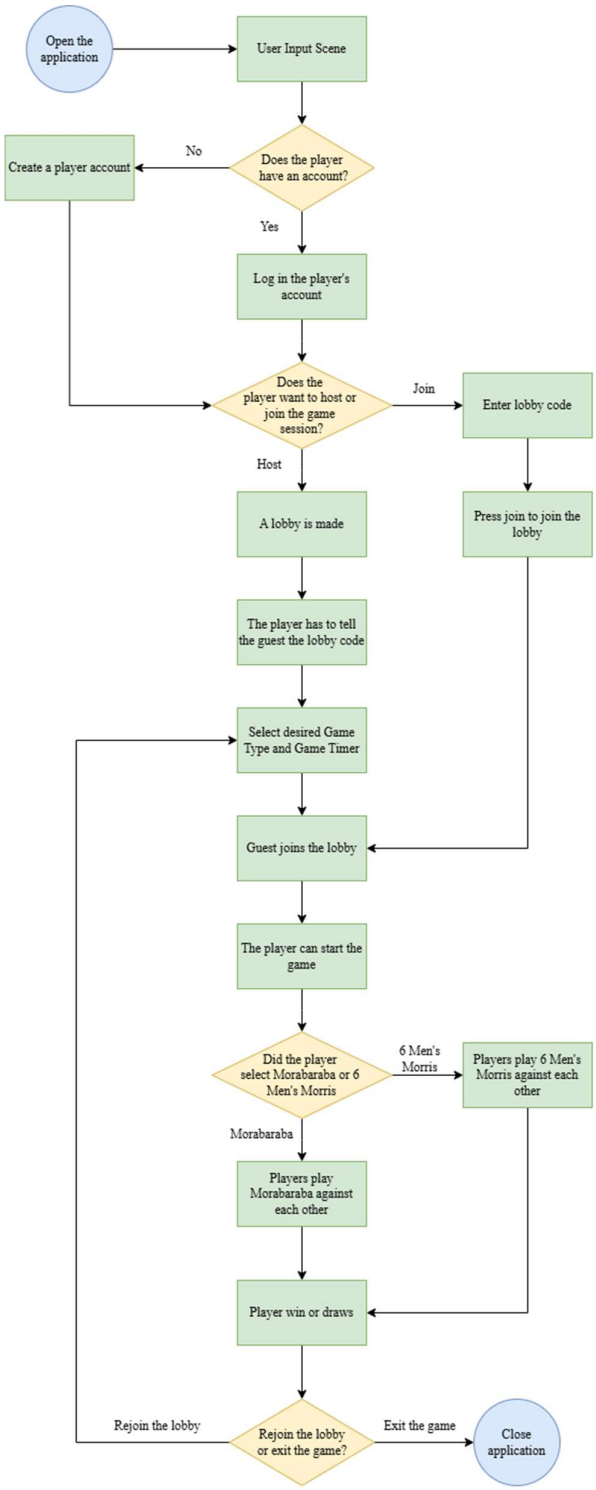
- **Toggl**

Toggl was used for keeping record of the time spent on this project, to ensure excessive time was not spent on menial tasks, as well as to hold team members accountable for the time they were spending on the project.

3. System architecture

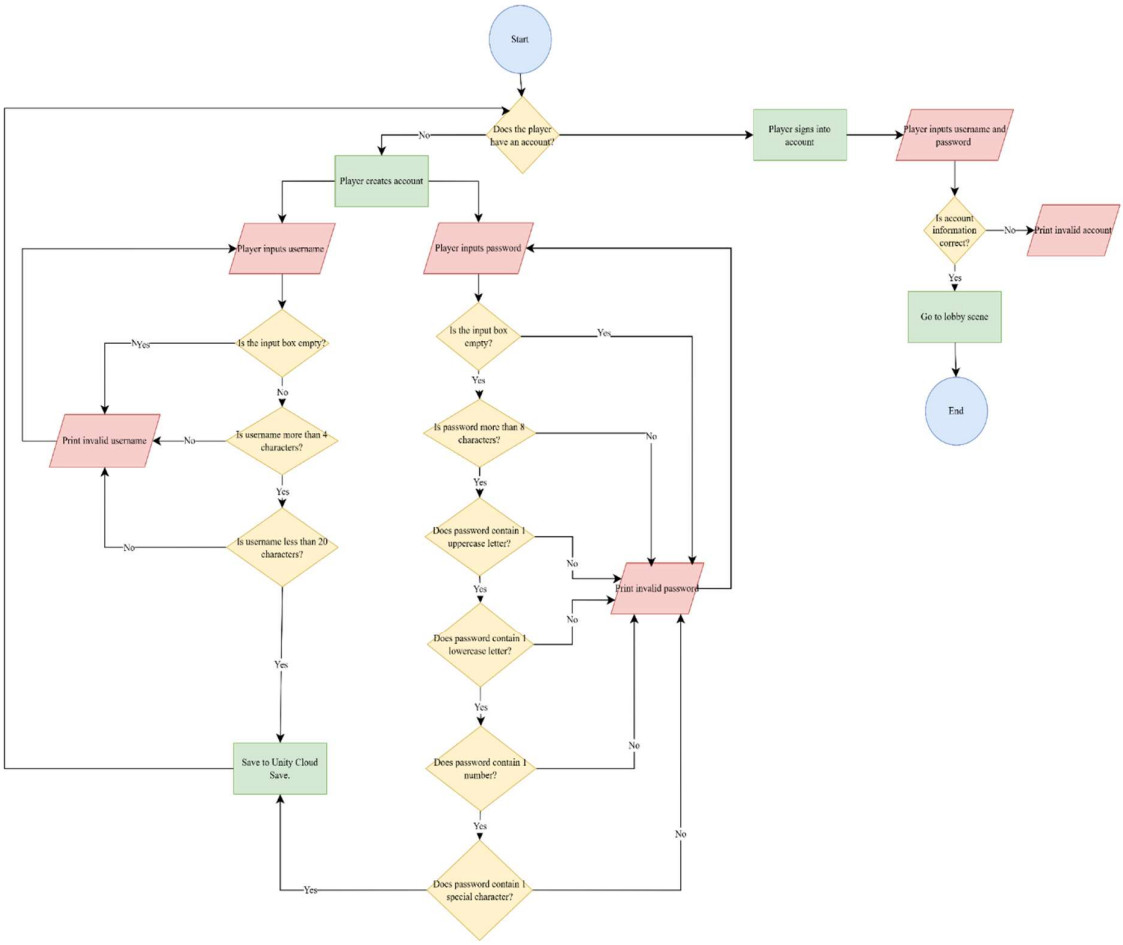
- **Overall game flow architecture**

As a whole, the project follows a logical flow through the different scenes, from user creation through to gameplay.



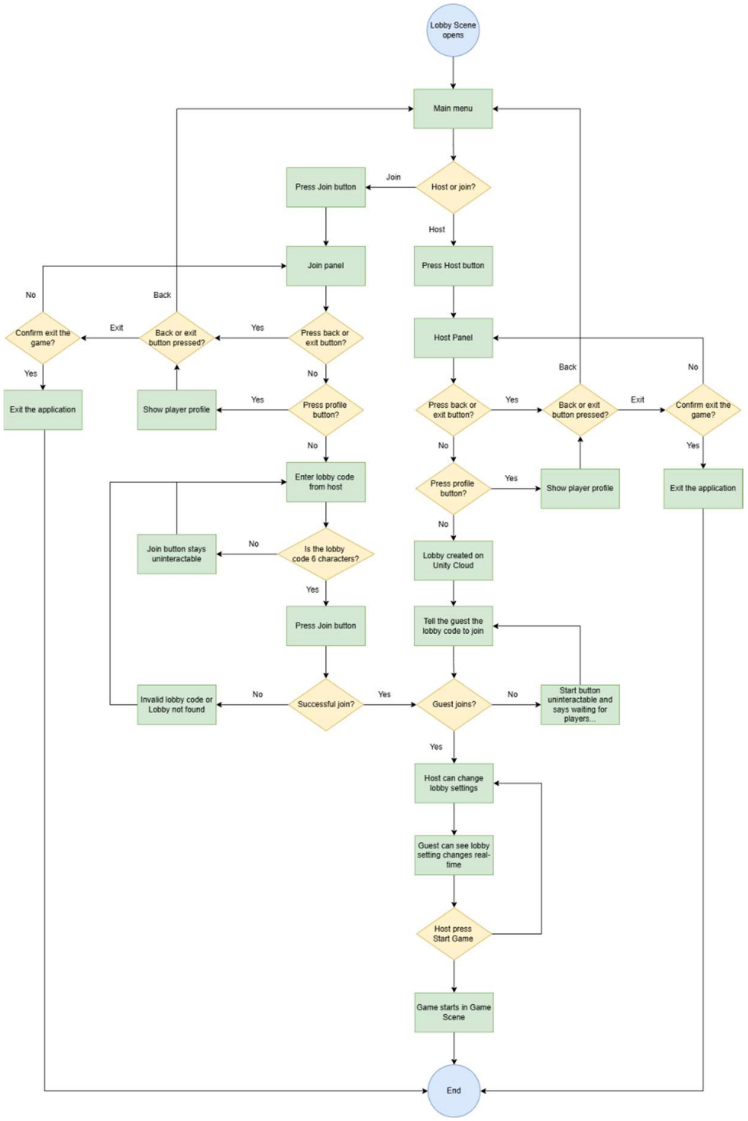
- **Sign-in system**

Players can create an account and log back into the account at any point.



- **Lobby system**

Players can dynamically create and join game sessions using unique lobby codes. The host creates the lobby on Unity Cloud, which generates an alphanumeric code for the guest. The guest submits the lobby code to query and join the matching cloud lobby successfully. Lobby metadata is synchronised between the host and the connected clients (guest), including the information about players in the lobby, game variant and timer chosen.

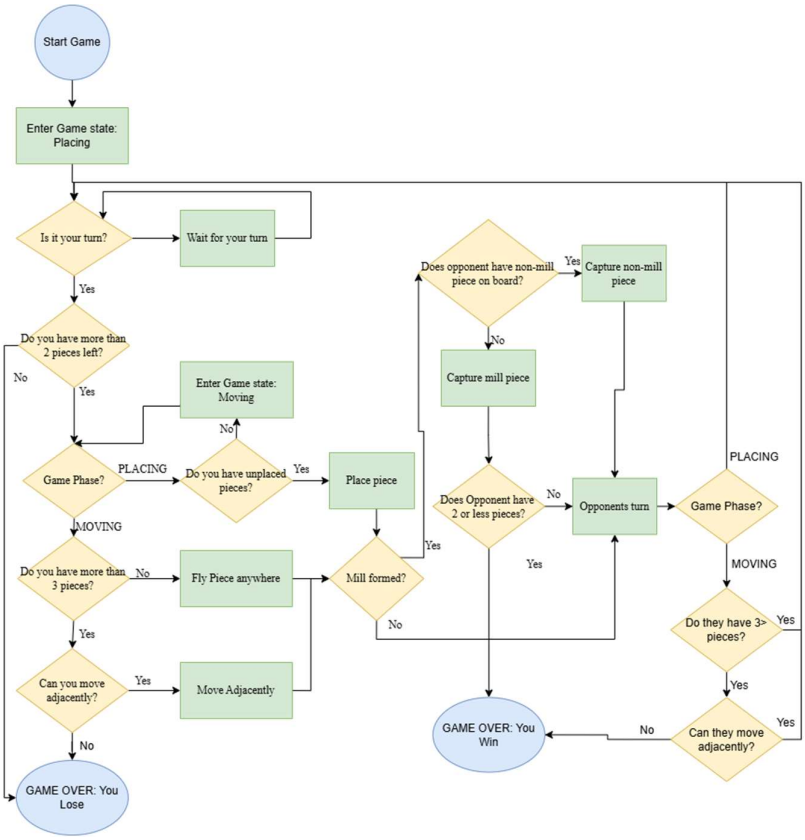


- **Multiplayer synchronisation**

Players' turn states had to be reliably synchronised across all connected clients for turn-based game logic in an online context. Network variable values are therefore used to detect which players' turn it is and the slot ID of the move they played. The game board input is enabled or disabled corresponding to this synchronised turn state where the player can only interact with the board when it is their turn.

- **Game state management**

To maintain a fair and consistent game state, the hosting device performs the move validation process where legal moves are committed to the authoritative game state to ensure all players see an identical and rule-compliant board at all times.



- **Timer system**

When the host creates the lobby, they have a choice of various game time options. They can choose either casual mode (no timer), turn times (each turn has a maximum time limit before it rolls over to the opponent's turn) or game time (there is a maximum overall time for the game to be completed before it results in a draw). This allows users to switch up the style of gameplay.

- **Rewind system**

The rewind functionality works by saving snapshots of the entire game state at important moments during gameplay, such as placing, moving, or capturing a piece. Each snapshot is stored inside a list and contains all critical information needed to restore the game back to that snapshot, including the current player, placement counter, pieces on the board, captures, current game phase, selected slot, and the ownership state of every slot on the board.

When a player presses the rewind button, the request is sent to the server, where the system first validates that it is the player's turn and that they still have rewinds available (only three are allowed per player, per game). The player's rewind count is then reduced, and the two most recent snapshots are used: the latest snapshot is discarded, while the previous snapshot is loaded to essentially undo the most recent turn. All saved values are returned to the game and the board visuals are reapplied so that every client sees the reverted game state.

4. Core systems

- **CreateUsername**

Users enter their username and password into their respective input boxes. It goes through a validation process to ensure the username and password meet certain requirements. When the user creates their account, they are directed to the lobby where they can immediately start. The username and password are saved in the Cloud Save. When the user wants to sign in again, they can log in using their details, and all their saved data loads. Validation is also put in place to ensure that they enter a valid and existing username and password.

- **PlayerData**

This is where the username, password, win count, lost count and draw count are kept to ensure they remain unchanged during scene changes. This is where the player statistics are changed during gameplay, and reflected in real time, and the data is uploaded to the Cloud Save.

- **AudioController**

This script controls any audio that plays throughout the duration of the game. It consists of a single function, whereby other scripts can call it with a unique string audio ID word and that specific sound will play.

- **GameController**

This script acts as the central game manager for the multiplayer implementation of Morabaraba and Six Men's Morris. It controls the full flow of each match, including player turns, piece placement and movement, mill detection, capturing mechanics, win conditions, timers, pausing, forfeiting, rewinding moves, and synchronisation of game states across all connected clients.

The script also manages the user interface by updating turn indicators, timers, capture counts, rewind counts, pause menus, and win/loss/draw screens, while calling audio and visual effects functions for actions such as placing pieces, forming mills, capturing, and rewinding. Additionally, it stores snapshots of previous game states to support a "rewind

round" feature, and uses network variables and RPCs to ensure all players see the same synchronised game board and play in real time.

- **SlotUI**

SlotUI controls the visualisation of ownership of each gameboard slot, either as player 1, player 2 or unoccupied. Furthermore, the script controls highlighting pieces when they are selected for movement, as well as resetting a slot back to unoccupied in the event of moving or capturing.

- **SlotID**

All information regarding the occupation status and mill status is controlled through this script. This allows other scripts to validate whether a slot is occupied or in a mill and thus perform the correct functionality in response to the validation. It also holds the ability to set a slot back to unoccupied, and is used when a piece is moved or captured.

- **GameTypeData**

This is a ScriptableObject that stores the specifications for the different game variants. Specifications include the name of the game variant, the total slots of the board belonging to that variant, and the number of pieces the players must start with.

- **UIManager**

This script handles all the logics for the lobby interfaces, such as joining and hosting, the creation of a lobby on Unity Cloud with an alphanumeric code generated, the connection of host and guest, as well as synchronising the lobby metadata (including selected game type, timer and players in lobby) between all connected clients.

5. Design decisions

When it comes to designing a game, the team has the most experience with **Unity**, which contains packages that helped with multiplayer and player account implementation.

Unity Netcode easily syncs all the game state, game objects and scenes across the host and connected clients. This greatly assists in the synchronisation of the game as a whole.

Unity Cloud keeps important Unity services centralised. This means the team can access services like Cloud Save and Player Authentication, and remotely manage and store the data that the player provides.

The client-host architecture was implemented using **NGO** and **Unity Relay**, as this reduces the financial costs of server hosting, while providing low-latency responsiveness for the core game logic.

In terms of coding practices such as data traversal, the game uses a predefined array-of-arrays structure to represent both mill patterns and adjacency relationships on the board, with each slot having a unique integer ID.

Mills are stored as fixed groups of three indices, while adjacency is stored as lists of directly connected slots that define legal movement options. This design was chosen because the board is static and rule-based, meaning all valid connections and winning conditions are fully defined in advance, rather than calculated dynamically. Using arrays for both mills and adjacency keeps the system simple, predictable, and easy to maintain.

Mill detection is performed by iterating through each predefined pattern and checking whether all slots in a given mill are occupied by the same player. For adjacency, valid moves are determined by checking whether a targeted slot exists within the current slot's adjacency list. In both cases, the logic relies on small, fixed datasets, meaning performance remains constant and does not degrade with different gameplay.

More complex approaches such as matrix-based representations were considered but ultimately rejected as it would introduce unnecessary code complexity for a board whose relationships never change, as well as the fact that the gameboards are not perfect squares, thus nullifying the use of a grid approach.

Overall, this approach ensures that both movement validation and mill detection remain straightforward, understandable for debugging, and functional.

6. Challenges encountered

Below, the challenges faced by each group member are discussed.

- **Akiria Chetty**

  The player statistics were not initially going to be stored in Cloud Save with the username and password. These stats were initially going to be stored using JSON data and a save file. This did not end up working however, because JSON stores data locally, which is good for single-player games, but not for an online multiplayer platform. JSON data can also be easily accessed by the player, who can then edit the data. Player statistics therefore had to be integrated into the Cloud Save system.

  Another challenge was trying to use the test cases in Unity. Rather than using C# in Visual Studio Code, Unity has its own built-in test runner that can be used to run test cases.

- **Karen Cheng**

  The multiplayer package version for this project was different from most guides and documentations [1] , which made it difficult to implement the multiplayer platform for this project with the correct multiplayer API functions and formats. The correct functions and API formats for Unity Lobby required extensive research, as the lobby metadata

synchronisation did not work using Unity Lobby functions [2]. Remote procedure calls were therefore implemented to guarantee the synchronisation.

- **Joanna Chronis**

Whilst the actual core gameplay of Morabaraba and Six Men's Morris did not pose any challenges during the single-device development phase, many bugs were created through multiplayer integration, since the complex relationship between client and server for playing a turn-based game posed a number of issues, such as duplicated moves, buggy visuals and, on occasion, client-side gameplay not updating at all. This added a great amount of time onto development, with each fix seemingly exposing new and more complex issues. Whilst this is not an actual challenge towards the basic logic of gameplay, this greatly affected the overall core gameplay portrayed to the players, and thus it was imperative that any and all bugs were fixed. All challenges and their solutions are discussed more indepth within the Individual Contributions report.

7. Testing suites

The test suites comprehensively covered all core systems necessary for the functionality of the online Morabaraba multiplayer platform. Each critical pathway within the entire platform was tested with both valid and invalid inputs to ensure robust error handling across the entire application.

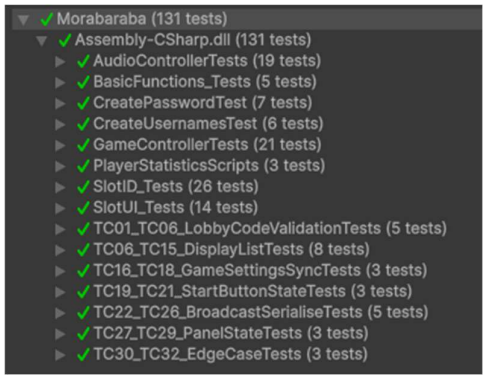
Beyond standard functionality tests, the suites also validated edge-case scenarios. These included negative time formatting, null and empty audio clip name handling, invalid player IDs, out-of-range slot numbers, and non-existent mill checks. Stress tests were also conducted to ensure the project could withstand a heavy command load.

The tests confirmed that the system properly uses all user inputs.

The 100% pass rate across all test cases provides high confidence that the application is stable, reliable, and ready for deployment. This comprehensive validation ensures that end users will experience a smooth, bug-free gaming experience with the assurance that all code works as required.

A comprehensive breakdown of all testing suites run to validate the functionality of the program is presented in Appendix A.

The Unity Test Runner output displayed below clearly depicts all test cases as passed, with no failures or errors recorded across any of the test suites.

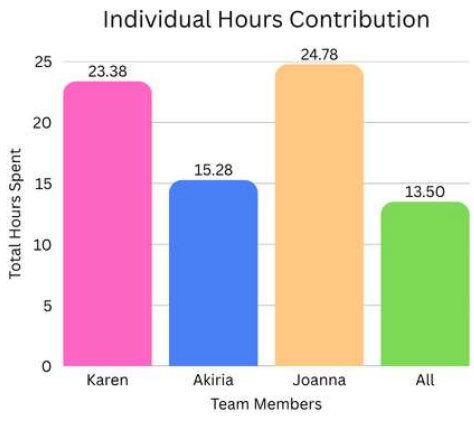


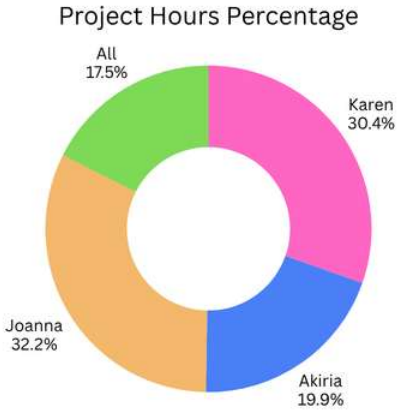
8. Total project time

Role	Expected time (hours)	Total time (hours)
General	6, plus meeting time and playtesting	13:29:27
Core game mechanics	40	24:46:30
Multiplayer platform and UI	40	23:23:02
Data analysis and statistics	40	15:17:32

The time estimated for how long the project would take was greatly over-estimated. It was estimated the entire project would take a total of 126 hours, but it ended up taking only 77 hours.

Each member spent significantly less time on the project due to conflicting schedules and other commitments. This, however, did not negatively impact the quality of work produced, as the project fully functions as needed and passes all test cases.





Data analytics and statics took the least amount of time, as this section of the design contains only the username authentication and player statistics. Core game mechanics, and the multiplayer platform and UI on the other hand, contained more complex code and interlinked states, thus requiring more time.

Despite this, however, all sections managed to remain below the estimated time. In-depth member time sheets are available in Appendix B, with clear visual representations of which member entered which entry.

9. Conclusion

The development of the ELEN3020A Group 3 Online Morabaraba Multiplayer Platform MVP successfully achieved the primary objective of delivering a fully functional, real-time online multiplayer experience for both Morabaraba and Six Men's Morris. The project fully implements a client-host architecture, providing secure gameplay. All core systems were implemented as planned, including account creation and authentication, persistent player statistics via Unity Cloud Save, dynamic lobby management with unique join codes, synchronised turn-based gameplay, mill detection, capture mechanics, timer functionality across multiple game modes, and a fully operational rewind system.

The comprehensive testing suites returned a 100% pass rate. This validates not only the correctness of the core game logic but also the robustness of edge-case handling, input validation, and system stability under stress conditions. This provides high confidence to the user that the platform is reliable and ready for deployment.

The project was completed in approximately 77 hours across all roles, coming in significantly below the estimated 126 hours. This efficiency did not compromise the quality of the final product however, as evidenced by the complete feature set and the perfect test pass rate. Overall, a fully functioning, fully networked, and thoroughly tested online multiplayer platform for traditional strategy games has been developed.

References

[1] Unity Technologies. 2026. *Multiplayer Services SDK*. Available at: https://docs.unity.com/en-us/mps-sdk (Accessed: 5 May 2026).

[2] Unity Technologies. 2026. *Update Lobby Data*. Available at: https://docs.unity.com/en-us/lobby/update-lobby-data (Accessed: 7 May 2026).

Game Asset and Audio References

MJType (2026) *Romantic Sunrise*. Available at: https://www.dafont.com/romantic-sunrise.font (Accessed: 9 May 2026).

Zhitna, A. (no date) *Wooden material, textured surface wood comic background in cartoon style. Wall panel for game UI design vector illustration*. Available at: https://www.vecteezy.com/vector-art/19547194-wooden-material-textured-surface-wood-comic-background-in-cartoon-style-wall-panel-for-game-ui-design-vector-illustration (Accessed: 9 May 2026).

user4468087 (no date) *Wooden buttons cartoon interface game UI elements*. Available at: https://www.magnific.com/premium-vector/wooden-buttons-cartoon-interface-game-ui-elements_30952419.htm (Accessed: 9 May 2026).

marsllzent (Freesound). (2022) *Place*. Available at: https://pixabay.com/sound-effects/film-special-effects-place-44638/ (Accessed: 4 May 2026).

floraphonic. (2024) *Minimal pop click UI 1*. Available at: https://pixabay.com/sound-effects/film-special-effects-minimal-pop-click-ui-1-198301/ (Accessed: 4 May 2026).

dishantbabariya. (2026) *Chalk slide short*. Available at: https://pixabay.com/sound-effects/film-special-effects-chalk-slide-short-516557/ (Accessed: 4 May 2026).

ALEXIS_GAMING_CAM. (2025) *Accepter 2*. Available at: https://pixabay.com/sound-effects/film-special-effects-accepter-2-394924/ (Accessed: 4 May 2026).

junioursoundays. (2026) *UI sound 03*. Available at: https://pixabay.com/sound-effects/film-special-effects-ui-sound-03-527847/ (Accessed: 4 May 2026).

litupsubway. (2026) *UI equip SFX*. Available at: https://pixabay.com/sound-effects/technology-ui-equip-sfx-513361/ (Accessed: 4 May 2026).

PW23CHECK. (2024) *Winning*. Available at: https://pixabay.com/sound-effects/film-special-effects-winning-218995/ (Accessed: 4 May 2026).

ShidenBeatsMusic. (2022) *No luck too bad disappointing sound effect*. Available at: https://pixabay.com/sound-effects/film-special-effects-no-luck-too-bad-disappointing-sound-effect-112943/ (Accessed: 4 May 2026).

DRAGON-STUDIO. (2025) *Simple whoosh*. Available at: https://pixabay.com/sound-effects/film-special-effects-simple-whoosh-382724/ (Accessed: 4 May 2026).

Appendices

Appendix A: Test case suites and their corresponding test cases

Create username test cases and results				
Test case	**Input**	**Expected result**	**Actual result**	**Status**
Empty username	“ ”	“Please enter a username”	“Please enter a username”	Pass
Short username	“ab”	“Username must be 3+ characters”	“Username must be 3+ characters”	Pass
Long username	“abcdefghijklmnopqrstuv”	“Username must be under 20 characters”	“Username must be under 20 characters”	Pass
Correct username	“Player”	Correct username	“Player”	Pass

Create password test cases and results				
Test case	**Input**	**Expected result**	**Actual result**	**Status**
Empty password	“ “	“Please enter a password”	“Please enter a password”	Pass
Short password	“Kiri1!”	“Password must be 8 characters or more”	“Password must be 8 characters or more”	Pass
No uppercase letter	“password1!”	“Password needs an uppercase letter”	“Password needs an uppercase letter”	Pass
No lowercase letter	“PASSWORD1!”	“Password needs a lowercase letter”	“Password needs a lowercase letter”	Pass
No number	“Password!!”	“Password needs a number”	“Password needs a number”	Pass

No special character	“Password11”	“Password needs a special character”	“Password needs a special character”	Pass
Correct password	“Password1!”	Correct password	“Password1!”	Pass

Player data test cases and results				
Test case	**Input**	**Expected result**	**Actual result**	**Status**
Set wins to value	5	Wins set to value	Wins set to 5	Pass
Set loss to value	3	Loss set to value	Loss set to 3	Pass
Set draw to value	2	Draws set to value	Draws set to 2	Pass

Lobby codes test cases and results				
Test case	**Input**	**Expected result**	**Actual result**	**Status**
Exact 6 character code accepted	“ABC123”	Accepted	Join proceeds	Pass
4 character code rejected	“AB12”	Rejected	Join button stays disabled	Pass
Empty string rejected	“ ”	Rejected	Join button stays disabled	Pass
Null input handled safely	Null	Returns false	No exception thrown	Pass
Lowercase normalised to upper	abc123	Accepted	ABC123	Pass

Display list test cases and results				
Test case	**Input**	**Expected result**	**Actual result**	**Status**
Host only entry	“Yuki|CID:0”	Display: [Yuki]	Display: [Yuki]	Pass
Host listed before guest	“Yuki|CID:0, Rene|CID:1”	Display: [Yuki, Rene]	Display: [Yuki, Rene]	Pass
Host ordering ignores raw order	“Rene|CID:1, Yuki|CID:0”	Display: [Yuki, Rene]	Display: [Yuki, Rene]	Pass
Malformed entry handled	“Yuki”	Display: []	Falls back with no crash and entry kept	Pass
Empty raw list	“ ”	Display: []	Empty display list	Pass
Username more than 20 characters	“VeryLongPlayerNameThatExceedsLimit”	Display: [VeryLongPlayerNameTh]	Display first 20 letters only	Pass
Null username	Null	Display: [Guest]	Stored as “Guest”	Pass

Empty username	“ ”	Display: [Guest]	Stored as “Guest”	Pass

Settings sync test cases and results				
Test case	**Input**	**Expected result**	**Actual result**	**Status**
Morabaraba maps to index 0	Morabaraba selected for Game Type dropdown	Dropdown index 0	Game type = Morabaraba	Pass
6 Men’s Morris maps to index 1	6 Men’s Morris selected for Game Type dropdown	Dropdown index 1	Game type = 6 Men’s Morris	Pass
Casual timer maps to index 0	Casual selected for Game Time dropdown	Dropdown index 0	Game time = Casual	Pass

Start button test cases and results				
Test case	**Input**	**Expected result**	**Actual result**	**Status**
1 player in lobby	Host in lobby	Button disabled	“Waiting for Players...”	Pass
2 players in lobby	Host and a guest in lobby	Button enabled	“Start Game”	Pass
Button text with 1 player in lobby	Player in lobby == 1	“Waiting for Players...”	“Waiting for Players...”	Pass

Broadcast test cases and results				
Test case	**Input**	**Expected result**	**Actual result**	**Status**
Single name serialises correctly	Serialise([Yuki])	Result: Yuki	Result: Yuki	Pass
2 names pipe-delimited	Serialise([Yuki, Rene])	Result: Yuki|Rene	Result: Yuki|Rene	Pass
Round-trip is lossless	Serialise then deserialise [Yuki, Rene, Karen]	Restored list equals original	Restored list equals original	Pass
Empty string	Deserialise(“”)	Returns []	Returns []	Pass
Null string	Deserialise(null)	Returns []	No exception	Pass

Panels test cases and results				
Test case	**Input**	**Expected result**	**Actual result**	**Status**
Show main menu	Show main menu	Returns to main menu	mainMenuPanel ON; others OFF	Pass

Leave lobby	Leave lobby	Returns to main menu	mainMenuPanel ON; hostingPanel OFF	Pass
Client disconnect	Disconnect	Returns to main menu	mainMenuPanel ON; joiningPanel OFF	Pass
Reset to defaults	Reset	Returns to main menu	mainMenuPanel ON; hostingPanel OFF	Pass
GetPlayerDisplayNames( ) returns copy	Returned list is modified externally	Internal list unchanged	Internal list unchanged	Pass
ClearLobbyData() empties player list	ClearLobbyData() called mid-lobby	rawList.Count == 0	rawList.Count == 0	Pass

AudioController test cases and results				
Test case	**Input**	**Expected result**	**Actual result**	**Status**
PlayAudio is case insensitive for “Select” audio	"select", "SELECT", "Select", "SeLeCt"	No exceptions thrown regardless of case	No exceptions thrown	Pass
PlayAudio is case insensitive for “Place” audio	"place", "PLACE", "Place", "pLaCe"	No exceptions thrown regardless of case	No exceptions thrown	Pass
PlayAudio is case insensitive for “Move” audio	"move", "MOVE", "Move", "mOvE"	No exceptions thrown regardless of case	No exceptions thrown	Pass
PlayAudio is case insensitive for “FormMill” audio	"formmill", "FORMMILL", "FormMill", "fOrMmIlL"	No exceptions thrown regardless of case	No exceptions thrown	Pass
PlayAudio is case insensitive for “BreakMill” audio	"breakmill", "BREAKMILL", "BreakMill", "bReAkMiLl"	No exceptions thrown regardless of case	No exceptions thrown	Pass
PlayAudio is case insensitive for “Capture” audio	"capture", "CAPTURE", "Capture", "cApTuRe"	No exceptions thrown regardless of case	No exceptions thrown	Pass
PlayAudio is case insensitive for “Win” audio	"win", "WIN", "Win", "wIn"	No exceptions thrown regardless of case	No exceptions thrown	Pass
PlayAudio is case-insensitive for “Loss” audio	"loss", "LOSS", "Loss", "lOsS"	No exceptions thrown regardless of case	No exceptions thrown	Pass

PlayAudio is case-insensitive for “Draw” audio	"draw", "DRAW", "Draw", "dRaW"	No exceptions thrown regardless of case	No exceptions thrown	Pass
PlayAudio is case-insensitive for “Rewind” audio	"rewind", "REWIND", "Rewind", "rEwInD"	No exceptions thrown regardless of case	No exceptions thrown	Pass
PlayAudio is case-insensitive for “Fly” audio	"fly", "FLY", "Fly", "fLy"	No exceptions thrown regardless of case	No exceptions thrown	Pass
PlayAudio is case-insensitive for “Invalid” audio	"invalid", "INVALID", "Invalid", "iNvAlId"	No exceptions thrown regardless of case	No exceptions thrown	Pass
PlayAudio shows a warning when clip is unknown	"UnknownClip", "nonexistent"	Warning logged: "Unknown audio request: X"	Warning containing "Unknown audio request" logged	Pass
PlayAudio shows a warning when name is empty	"" (empty string)	Warning logged: "Audio clip name is null or empty"	Warning containing expected message logged	Pass
PlayAudio shows a warning when name is null	null	Warning logged: "Audio clip name is null or empty"	Warning containing expected message logged	Pass
All Clips are assigned	N/A (initialization)	All 12 AudioClip fields are not null	All clips assigned and not null	Pass
A null audioclip does not throw an exception	WinSound set to null, then PlayAudio("Win")	No exception thrown when clip is null	No exception thrown	Pass
Sequential audio calls work	Play 6 different audio calls sequentially	No exceptions thrown during sequence	No exceptions thrown	Pass
Rapic call stress test	100 iterations of mixed case audio calls	No exceptions thrown under load	No exceptions thrown	Pass

BasicFunctions test cases and results				
Test case	**Input**	**Expected result**	**Actual result**	**Status**
OnClose activates object on first call	Call onClose() once	closeTarget GameObject becomes active	Object activated successfully	Pass
call OnClose deactivates object on first call	Call onClose() twice	closeTarget GameObject becomes inactive	Object deactivated successfully	Pass
OnClose correctly toggles	Call onClose() 5 times (odd number)	closeTarget GameObject is active	Object active after odd number of toggles	Pass

OnClose doesnt crash when closetarget is null	CloseTarget = null, then call onClose()	No exception or null reference error	No exception thrown	Pass
QuitGame doesnt crash	Call QuitGame()	No exception thrown during quit attempt	No exception thrown	Pass

GameController test cases and results				
Test case	**Input**	**Expected result**	**Actual result**	**Status**
SetSlotOwner sets correct owner to valid slot	SetSlotOwner(1, 1)	GetSlotOwner(1) returns 1	Owner correctly set to 1	Pass
Empty slot returns zero	SetSlotOwner(1, 1)	Returns 0	Returned 0	Pass
New owner replaces old owner	SetSlotOwner(1, 1)	Final owner is 2	Owner successfully overwritten to 2	Pass
Neighboring slots are adjacent	IsAdjacent(slot1, slot2)	Returns true	Returned true	Pass
Far slots aren't adjacent	IsAdjacent(slot1, slot10)	Returns false	Returned false	Pass
Moving next door is allowed	Move from slot 1 to 2 (adjacent, target empty)	Returns true	Returned true	Pass
Jumping far away is forbidden	Move from slot 1 to 5 (not adjacent)	Returns false	Returned false	Pass
Can't move onto an occupied slot	Move to slot already owned by opponent	Returns false	Returned false	Pass
Three in a row makes a mill	Slots 1,2,3 owned by Player 1	CheckMill(3, 1) returns true	Returned true	Pass
No mill without three in a row	Slots owned: 1=P1, 2=P2, 3=P1	CheckMill(3, 1) returns false	Returned false	Pass
Two pieces don't make a mill	Slots 1=P1, 2=P1, 3=empty	CheckMill(2, 1) returns false	Returned false	Pass
Valid slot number finds the slot	GetSlotByNumber(1)	Returns non-null Slot with number 1	Slot returned correctly	Pass
Invalid slot number returns nothing	GetSlotByNumber(999)	Returns null	Returned null	Pass
Zero seconds shows 00:00	FormatTime(0)	Returns "00:00"	Returned "00:00"	Pass
Five seconds shows 00:05	FormatTime(5)	Returns "00:05"	Returned "00:05"	Pass

Ninety seconds shows 01:30	FormatTime(90)	Returns "01:30"	Returned "01:30"	Pass
Negative time shows 00:00	FormatTime(-10)	Returns "00:00"	Returned "00:00"	Pass
Game saves a backup	Call SaveSnapshot()	History count increases by 1	Count incremented by 1	Pass
Turn ends, other player goes	Current player = 1, call EndTurn()	Current player becomes 2	Player switched to 2	Pass
Many slot changes won't crash	Loop setting and getting all 24 slots	No exceptions thrown	No exceptions thrown	Pass
Many mill checks won't crash	Loop checking mills for slots 1-24 for both players	No exceptions thrown	No exceptions thrown	Pass

SlotID test cases and results				
Test case	**Input**	**Expected result**	**Actual result**	**Status**
New slot starts empty	New SlotID component	IsOccupied = false, occupiedBy = 0	Both assertions passed	Pass
New slot starts not in a mill	New SlotID component	isInMill = false	Assertion passed	Pass
Awake finds the slot UI automatically	New Slot with SlotUI child	slotUI reference is auto-assigned	Reference assigned correctly	Pass
Player 1 correctly occupies the slot	SetOccupant(1)	occupiedBy = 1, IsOccupied = true	Both assertions passed	Pass
Player 2 correctly occupies the slot	SetOccupant(2)	occupiedBy = 2, IsOccupied = true	Both assertions passed	Pass
Player 1 sets correct colour	SetOccupant(1)	occupiedBy = 1	Assertion passed	Pass
Player 2 sets correct colour	SetOccupant(2)	occupiedBy = 2	Assertion passed	Pass
Zero sets empty color	Set to 1, then SetOccupant(0)	occupiedBy = 0, IsOccupied = false	Both assertions passed	Pass
Negative player number won't crash	SetOccupant(-1)	No exception thrown	No exception thrown	Pass
Invalid player number won't crash	SetOccupant(99)	No exception thrown	No exception thrown	Pass
Changing players updates correctly	Set to 1, then to 2	occupiedBy = 2, IsOccupied = true	Both assertions passed	Pass

Clearing a player 1 slot empties it	Set to 1, then ClearSlot()	occupiedBy = 0, IsOccupied = false	Both assertions passed	Pass
Clearing a milled slot removes mill status	Set mill=true, then ClearSlot()	isInMill = false	Assertion passed	Pass
Clearing resets slot color to empty	Set to 1, then ClearSlot()	occupiedBy = 0	Assertion passed	Pass
Clearing an empty slot does nothing	ClearSlot() on already empty slot	occupiedBy=0, IsOccupied=false, isInMill=false	All assertions passed	Pass
Empty slot says not occupied	Default state	IsOccupied = false	Assertion passed	Pass
Player 1 slot says occupied	occupiedBy = 1	IsOccupied = true	Assertion passed	Pass
Player 2 slot says occupied	occupiedBy = 2	IsOccupied = true	Assertion passed	Pass
Occupied status changes correctly over time	Change: empty → P1 → clear	IsOccupied = false → true → false	All states correct	Pass
Marking a slot as mill returns true	SetMillStatus(true)	isInMill = true	Assertion passed	Pass
Removing a slot from mill returns false	Set true, then SetMillStatus(false)	isInMill = false	Assertion passed	Pass
Mill status toggles correctly with multiple calls	true → false → true	isInMill = true → false → true	All toggles correct	Pass
Slot number can be assigned	slotNumber = 5	slotNumber = 5	Assertion passed	Pass
Slot number supports up to 24	slotNumber = 24	slotNumber = 24	Assertion passed	Pass
Empty to occupied to cleared keeps correct state	Cycle: empty → P1 → mill → clear	Final: occupiedBy=0, isInMill=false	Both assertions passed	Pass
Changing occupant does not affect mill status	Set P1, mill=true, then set to P2	isInMill remains true, occupiedBy=2	Both assertions passed	Pass

SlotUI test cases and results				
Test case	**Input**	**Expected result**	**Actual result**	**Status**
Slot UI needs an image component	Check class attributes	RequireComponent attribute present	Attribute found	Pass
Slot UI needs an image component	SetPlayerColor(1)	Slot image color = player1Color	Colours match	Pass

Player 2 sets correct colour	SetPlayerColor(2)	Slot image color = player2Color	Colours match	Pass
Empty or invalid sets empty colour	SetPlayerColor(0), SetPlayerColor(3), SetPlayerColor(-1)	Slot image color = emptyColor for all	Colours match for all inputs	Pass
Highlight for Player 1 works	Highlight(1)	Slot image color = highlightplayer1Color	Colours match	Pass
Highlight for Player 2 works	Highlight(2)	Slot image color = highlightplayer2Color	Colours match	Pass
Invalid player highlight shows empty	Highlight(0), Highlight(99)	Slot image color = emptyColor for both	Colours match for all inputs	Pass
Mill highlight for Player 1 works	HighlightMill(1)	Slot image color = millplayer1Color	Colours match	Pass
Mill highlight for Player 2 works	HighlightMill(2)	Slot image color = millplayer2Color	Colours match	Pass
Invalid mill highlight shows empty	HighlightMill(0)	Slot image color = emptyColor	Colours match	Pass
Reset colour returns to empty	Set to P1 color, then ResetColor()	Slot image color = emptyColor	Colours match	Pass
All colour properties are assigned	Check all 7 color properties	All 7 colors are not null	All colours assigned	Pass
Colour changes work in sequence	P1 color → Highlight → HighlightMill → Reset	Each step produces correct color	All colour changes correct	Pass
Colours persist when switching players	P1 → P2 → Mill P1 → Highlight P2	Each color change applies correctly	All colour changes correct	Pass

Appendix B: Time sheets

Date	Member	Description	Duration (hh:mm)
Mar 4, 2026	All	Group Meeting	00:16
Mar 4, 2026	All	Creating Repository	00:19
Mar 5, 2026	Karen	Unity Relay code research	00:56
Mar 5, 2026	Joanna	Researching Tile Recognition	00:17
Mar 10, 2026	Karen	Testing Unity Packages	01:33
Mar 11, 2026	All	Group Meeting	00:13
Mar 12, 2026	Akiria	Research JSON	01:25
Mar 14, 2026	Akiria	Made UI	00:47
Mar 16, 2026	Karen	Code-based Lobby Code Research	00:45
Mar 16, 2026	Joanna	Creating base of game	02:10
Mar 18, 2026	All	Group Meeting	00:50
Mar 25, 2026	Akiria	Redo UI input	00:33
Apr 1, 2026	Joanna	Base Mechanics for Single Player	00:28
Apr 2, 2026	Joanna	Game Controller planning	00:24
Apr 6, 2026	Joanna	Movement and mill creation	02:34
Apr 6, 2026	Joanna	Illegal move registering	00:25
Apr 6, 2026	Joanna	Win Conditions	01:16
Apr 8, 2026	All	Playtesting core mechanics	00:29
Apr 15, 2026	All	Group Meeting	00:43
Apr 20, 2026	Karen	Implement Unity Services	01:44
Apr 21, 2026	Karen	Attempted merge of game mechanics and Unity Services	01:24
Apr 22, 2026	Karen	Created Lobby UI scene	01:26
Apr 22, 2026	All	Group Meeting	00:19
Apr 22, 2026	Akiria	Implementes a create username	01:44
Apr 28, 2026	All	Group Meeting	00:21
Apr 29, 2026	Karen	Synchronising lobby metadata	03:27
May 2, 2026	Karen	Connection of players in gameplay	04:14
May 4, 2026	Karen	Synchronisation of player moves	03:32
May 4, 2026	All	Group Meeting	01:24
May 5, 2026	Karen	Minor fixes of synchronisation	01:47
May 5, 2026	Joanna	Audio Integration - Gameplay	03:41
May 5, 2026	Akiria	Planning code	01:13
May 6, 2026	Joanna	Fixing bugs	00:38
May 6, 2026	Joanna	Depicting captures and pieces left	02:41
May 6, 2026	Akiria	Added a saves wins and loss script	00:57
May 6, 2026	Akiria	New username and password scripts	01:42
May 7, 2026	Joanna	Fixing return to lobby button	01:08

May 7, 2026	Joanna	Rewind Mechanic	00:26
May 7, 2026	Joanna	Timer Mechanics	01:20
May 7, 2026	Akiria	New username and password scripts	01:45
May 8, 2026	Karen	Unified all UIs in the game	02:35
May 8, 2026	All	Playtest	00:57
May 8, 2026	Joanna	Game Statistic showing	00:18
May 8, 2026	Joanna	Creating Six Mens Morris variant logic	01:59
May 8, 2026	Akiria	Add draw, fixed username and passwords, added a stats UI	02:07
May 8, 2026	Akiria	Trying to fix username issues	01:26
May 9, 2026	All	Playtest	01:34
May 9, 2026	All	Document Writing	01:56
May 9, 2026	Joanna	Fixing Bugs	02:52
May 10, 2026	All	Document Writing	04:09
May 10, 2026	Joanna	Test Cases	02:13
May 10, 2026	Akiria	Fixing error messages, added code comments and tes cases	01:37